C Sharp and Java Benchmark Plan

An AI implementation contract for independent performance evaluation

Purpose and scope

Prepared for Michael on 25 October 2026. Specification version 1.0. This is an implementation and measurement contract for an AI agent, with a proposed ranking model. No benchmarks have been executed for this plan, and no performance results are asserted.

Implement equivalent C# and Java applications and measure the complete set of performance domains below. The mandatory database matrix uses SQL Server, PostgreSQL, and SQLite in memory, with both entity ORM access and direct parameterized SQL in each language. The wider suite measures computation, collections, memory and garbage collection, concurrency, serialization, file and network I/O, HTTP services, startup, deployment, native interop, and selected language mechanisms.

Exclude popularity, employment demand, and IDE quality. Treat the user's preference for C# as irrelevant to implementation, measurement, and scoring. Report advantages for either implementation when the evidence supports them.

Performance is a property of an implementation running on a particular compiler, runtime, library, operating system, and machine. These tests compare those stacks under a common contract. They cannot isolate language syntax from all runtime effects or establish that a language is universally better. Expressiveness, framework breadth, ecosystem coverage, licensing, maintainability, and safety guarantees also require qualitative evidence; elapsed time cannot stand in for those evaluations.

The Markdown file is the authoritative AI handoff. The PDF and Word versions contain the same specification. The accompanying JSON is a configuration starting point, not a runnable benchmark implementation. Optional profiles must be reported separately from the mandatory baseline.

1 Instructions for the implementation AI

Act as an independent benchmark engineer. Implement both sides of this contract in one repository. Read the complete specification before selecting dependencies or writing code. Do not optimize for a preferred winner.

1. Emit a resolved version manifest, a case inventory, and a semantic parity checklist before running measurements. Resolve every placeholder to an exact supported version or explain why a cell is blocked.

2. Implement the shared data generator, correctness oracle, trace files, schemas, queries, both applications, execution controller, result validator, and report generator. Keep shared inputs authoritative and independent of either language implementation.

3. Build both applications and run semantic tests and smoke runs. Implement all mandatory cases. Mark a missing or unsupported case explicitly; never replace an engine, ORM, workload, durability guarantee, or runtime silently.

4. Run a pilot to determine warmup behavior, trace capacity, load-generator capacity, disk and memory requirements, and estimated full-suite duration. Freeze the run manifest and weights after the pilot and before the scored campaign.

5. Provide a dry-run command and a complete full-run command. A smoke run or pilot is not a ranking result. Execute the full campaign only when the task's execution scope and resource budget include it; never fabricate measurements when execution is unavailable.

6. Produce raw evidence, a reproducible report, coverage status, paired comparisons, and the weighted results defined here. Include nulls and reasons for unavailable metrics.

7. Keep implementation choices symmetric. Any tuning change requires a new named profile and rerunning the affected pair. Archive the original results and explain the change.

8. Use disposable benchmark resources and clearly named benchmark databases. Scripts must reject unlisted remote hosts and must not reset an arbitrary database supplied through a connection string. Keep credentials out of output. Document SQL Server edition, applicable terms, and operator EULA acceptance rather than silently accepting terms for someone else.

Expected first response from the AI: a short architecture description, the resolved compatibility manifest, the full case inventory, implementation steps, blockers, and commands to reproduce the work. Continue implementing within the authorized task; an architecture description alone is not completion.

2 Comparison tracks and version policy

The requested feature-generation track is C# 15 on a pinned .NET 11 SDK/runtime and Java 27 on a pinned JDK distribution. As of this specification date, Microsoft's C# 15 documentation identifies .NET 11 as prerelease, while JDK 27 is generally available. Label that maturity difference in every report. Recheck release status and compatibility at implementation time. [1, 2]

Pin SDK, runtime, compiler language level, JDK vendor/build, ORM, provider, JDBC drivers, connection pool, JSON library, benchmark harness, server versions, container digests, OS image, and native libraries. Save lockfiles and SHA-256 hashes. Never use a floating latest tag in a measured campaign. SQLite wrapper versions alone do not identify the embedded engine: record sqlite_version(), sqlite_source_id(), compile_options, architecture, and native library hash from both processes.

Use supported dependency combinations verified by compilation and functional tests. If a provider does not support the chosen preview/runtime combination, report that gap and create a clearly labeled alternative track. Do not silently downgrade only one side. An optional stable corroboration track may use .NET 10 and JDK 25 with compatible libraries; it must have separate results and scores.

Track	Implementation contract	Reporting rule
Controlled JIT baseline	Same algorithms, inputs, operation semantics, resource budgets, fixed worker counts; normal optimized JIT execution	Primary paired results
Idiomatic application	Each stack uses its normal production design, including C# async and Java virtual threads for blocking I/O	Separate application results; identify framework and scheduler effects
Tuned variants	Compiled ORM queries, batching, specialized collections, source generation, explicit SIMD, or collector tuning	Equal tuning opportunity; no substitution into baseline
Native executables	.NET Native AOT versus GraalVM Native Image, where the same workload is supported	Separate deployment comparison; include build cost and feature limitations [12, 13]

Use preview features only in labeled cases that require them. Do not enable all Java preview features or C# experimental flags by habit. Keep server GC/tiered compilation and the JVM collector/JIT settings recorded. Equal CPU and process memory limits are the primary resource constraint; equal managed heap sizes alone are not

equal total memory. Add a fixed-heap sensitivity profile only when both heaps and native headroom are explicitly budgeted.

3 Mandatory database matrix

Entities means actual mapped entities managed by the ORM for entity workloads. C# uses Entity Framework Core and Java uses Hibernate ORM through Jakarta Persistence or its stateful Session API. Direct query means ADO.NET or JDBC, with hand-written, parameterized SQL and equivalent result mapping. A micro-ORM is not the raw baseline.

Database	C# entities and raw access	Java entities and raw access
SQL Server	EF Core SQL Server provider; Microsoft.Data.SqlClient	Hibernate SQL Server dialect; Microsoft JDBC Driver for SQL Server
PostgreSQL	EF Core Npgsql provider; Npgsql	Hibernate PostgreSQL dialect; PostgreSQL JDBC driver
SQLite in memory	EF Core SQLite provider; Microsoft.Data.Sqlite	Hibernate community SQLiteDialect; Xerial SQLite JDBC

This is 12 mandatory stack cells: two languages times three engines times two access modes. Every cell runs D01 through D08. Each engine has its own paired language results; a SQL Server result must never be paired against a PostgreSQL result to claim a language advantage.

Hibernate's SQLite dialect is a community dialect, with a different support boundary from its core dialects. Verify the exact released dialect/driver combination against this schema and all eight workloads. If it cannot implement a contract, mark that cell unsupported and preserve the gap. Do not use H2 as an unannounced replacement. [3]

In memory engine rules

Use the same SQLite engine version and, where feasible, identical compile options/native build in both processes. If exact native parity cannot be obtained, disclose the differences and qualify those results as wrapper/build comparisons. Exclude materially different builds from the primary aggregate until parity is resolved.

Use a named true in-memory database shared only among connections inside one process, such as the SQLite URI file:bench?mode=memory&cache=shared, translated to each provider's connection-string syntax. Keep one anchor connection open for the lifetime of a repetition. Both applications load identical canonical data into their own fresh database. They do not share the same memory database across processes. Plain :memory: creates a distinct database per connection. [4]

Foreign keys must be enabled on every connection. Use read_uncommitted=0, a documented journal mode appropriate to in-memory SQLite, and an explicitly recorded busy timeout. Do not claim disk durability or WAL behavior for the memory profile. Run the mandatory SQLite matrix at one worker with one leased connection, avoiding a shared-cache locking contest. Optional multi-connection read and contention profiles use the same settings and pool sizes on both sides and are scored separately.

EF Core's InMemory provider and Java H2 are excluded from the relational baseline because they are different engines with different behavior. They may be additional testing-tool demonstrations, with no C# versus Java database score. SQLite is a real relational engine but does not reproduce all SQL Server or PostgreSQL behavior. Microsoft explicitly cautions against treating substitute databases as proof of production-database correctness. [5]

Microsoft.Data.Sqlite executes asynchronous ADO.NET methods synchronously. The memory profile therefore uses synchronous APIs on both sides. Do not present an async method name as evidence of asynchronous SQLite I/O. [6]

4 Environment and resource contract

Prefer a dedicated native Linux x86-64 host for the primary SQL Server container campaign, after checking current vendor support. An example allocation is 16 physical cores and 64 GiB RAM: four isolated cores and a 4 GiB process/container limit for the application, eight different cores and 16 GiB for the database, and four different cores for the controller and load generator. Record physical versus logical cores, SMT placement, CPU model and microcode, NUMA layout, memory speed, kernel, container engine, filesystem, storage model, and cgroup limits. A smaller host is valid if the manifest specifies a non-overcommitted layout.

Run only one language application and one database engine at a time. Reuse the same database image, configuration, storage volume class, network path, and CPU allocation for each language pair. Run SQL Server and PostgreSQL campaigns separately. Use the same explicit network encryption policy for both clients of an engine, including certificate validation. TLS-off isolated-lab and TLS-on production profiles must have separate labels.

SQLite runs inside the application process; its engine and dataset count against the application's memory budget. Network database memory lives in another process. Report application RSS, database RSS, and total system cost separately. Do not compare these different memory boundaries as if they were identical deployment models.

Fix CPU governor/power mode, container limits, affinity, and storage caching policy. Record thermal throttling, CPU steal, page faults, disk latency, packet loss, and background activity. Never compare ARM emulation with a native x86 process. Repeat portability cases on additional operating systems/architectures with the same within-platform discipline; those are separate campaigns.

Use optimized Release builds with no debugger. Precompile before timed runs. Record JIT, tiering, dynamic PGO, GC mode, JVM flags, and application environment variables. Use the runtime's normal supported collector in the baseline, then disclose collector effects through separately budgeted sensitivity runs. BenchmarkDotNet's guidance also stresses optimized builds, consumed outputs, and conclusions limited to the measured environment. [7]

5 Canonical inputs and correctness oracle

One language-neutral generator produces the shared data, traces, expected outputs, and SHA-256 manifest. Python is an acceptable generator. Do not independently reproduce a pseudorandom sequence in C# and Java and assume it matches. All seeds, algorithms, encodings, and parameter distributions must be fixed in checked-in configuration. Use trace seed 20261025 unless intentionally changed before the campaign.

Use UTF-8, UTC epoch milliseconds, deterministic ASCII identifiers, signed 64-bit IDs, integer cents, and explicit ordinal/binary comparison for the database baseline. This avoids accidentally comparing different decimal translations, date precisions, collations, and locale behavior. Test richer text and decimal semantics separately. Validate string length, null handling, numeric ranges, row ordering, and overflow policy.

Dataset	Customers	Products	Orders	Order lines
S mandatory	10,000	1,000	100,000	500,000
M optional	100,000	10,000	1,000,000	5,000,000

Each initial order has exactly five lines. Generate uniform customer/product assignment and four deterministic order statuses with sufficiently populated query windows. Use contiguous client-assigned positive IDs and a disjoint high-ID range for measured inserts. Precompute per-worker write partitions and expected outcomes. The generator must also produce separate, equivalent delete populations for warmup and measurement.

The baseline uses a uniform read distribution and nonconflicting write partitions. Add a separate 80/20 hot-key distribution and intentional conflict profile. Write traces must not accidentally make the second language inherit a different database size, depleted delete pool, or warmer logical working set. Size each finite write trace conservatively in the pilot and abort as invalid if it exhausts; never wrap around into duplicate inserts. A time-based read run can consume different trace prefixes in the two languages; validate each executed prefix against the oracle rather than requiring identical operation counts or final checksums after unequal work.

Use an independent oracle to validate ordered outputs and committed final state. Tests must include row counts, foreign keys, totals, order-line quantities, expected version increments, duplicate keys, stale updates, rollback, non-ASCII serialization, and numeric boundary cases. Performance loops consume outputs through a lightweight checksum or harness blackhole; full cryptographic hashing and detailed correctness comparisons occur outside timing on both sides. For byte checksums, define the same unsigned modulo-2^64 operation explicitly, including C# unchecked arithmetic.

6 Database schema and SQL contract

Create schema from shared engine-specific DDL, not competing ORM auto-generated migrations. Map the same logical constraints and indexes into all three engines. SQLite uses INTEGER for signed 64-bit values; SQL Server/PostgreSQL use their corresponding explicit numeric types. Save the realized DDL and catalog inspection as evidence.

Table	Required fields	Constraints and indexes
Customer	id, email, name, balance_cents, version	Primary key id; unique email; all fields required
Product	id, sku, name, price_cents	Primary key id; unique sku; all fields required
Orders	id, customer_id, status, created_epoch_ms, total_cents, version	Required customer FK; indexes on customer_id/created_epoch_ms/id and status/created_epoch_ms/id
OrderLine	id, order_id, line_no, product_id, quantity, unit_price_cents	Required order and product FKs; unique order_id/line_no; product_id index

Disable cascade deletes for the baseline and delete dependents explicitly. Define email/sku length and ASCII/binary comparison explicitly. Add equivalent checks for positive quantity, nonnegative price, and permitted statuses where supported, with application validation equally applied. No extra covering index, denormalized column, trigger, cache, stored procedure, or hidden computed field may favor one language.

Use an explicit integer version column as an optimistic concurrency token. EF marks it as a concurrency token and increments it in application logic; Hibernate maps a numeric @Version with the same initial value and increment. Do not compare SQL Server rowversion with a different PostgreSQL/SQLite mechanism. All raw updates use the same old-version predicate and affected-row check.

The raw SQL text for a given engine comes from shared files. Only parameter marker syntax and unavoidable provider binding syntax may differ across languages. Enumerate columns; do not SELECT *. Bind explicit numeric

and string types and lengths. Materialize the same fields into equivalent objects, consume all results, and perform the same graph assembly. A scalar raw query is not the comparator for an ORM entity graph.

ORM SQL need not be textually identical to raw SQL, but it must do the same logical work. Save generated SQL, parameter types, plans, command counts, result row counts, and representative execution statistics from untimed diagnostic runs. Differences in optimizer plans or protocol behavior are evidence to explain, not a reason to conceal or manually rewrite only one side. Index use, projection, tracking, and related-entity loading are material measurement variables. [8]

7 Database workload definitions

One logical operation includes connection lease or acquisition, fresh unit of work if applicable, transaction work, complete result materialization, commit where required, and resource disposal. Factories, mapping metadata, seed/import, schema setup, and trace creation remain outside steady-state timing. Cold initialization is measured separately.

ID	Workload and returned value	Required database work
D01	Customer by primary key; full customer object	One parameterized SELECT; 100 percent valid IDs
D02	Keyset page of 50 orders by status and created/id cursor	One SELECT; explicit deterministic ordering and equal selected fields; no deep OFFSET
D03	One order with customer and five order lines	One joined SELECT; reconstruct one graph; no lazy loads or hidden N+1 queries
D04	Count and sum total_cents by status in a fixed date range	One grouped SELECT returning four ordered groups; ORM LINQ/HQL projection, not native SQL
D05	Update a customer balance by a fixed delta with optimistic version check	Explicit transaction; one SELECT, one version-qualified UPDATE, then commit
D06	Insert an order and five lines	Explicit transaction; six logical INSERT statements, then commit; client-assigned IDs
D07	Delete a preexisting order and its five lines	Explicit transaction; load graph once, five child DELETE statements, one version-qualified parent DELETE, commit
D08	Mixed business stream	60 percent D01, 15 percent D02, 10 percent D05, 10 percent D06, 5 percent D07; fixed precomputed proportions

D04 is an ORM query/projection case rather than a tracked-entity case by its nature. Label that distinction. D05 raw access must perform the read-modify-write transaction, rather than substituting one atomic UPDATE for two ORM statements. D06 and D07 must expose the same logical row modifications even if ORM SQL adds value-returning clauses for concurrency checks. Count actual round trips separately from logical statements; ORM transaction scaffolding may differ and remains visible in the measured stack.

D03 must configure single-query eager loading on both ORMs. Confirm graph equivalence and command counts before timing. D07 reads and deletes a population reserved solely for this workload. D08 has independent populations and preserves its intended proportions even if a workload fails; failed operations count as failures rather than being replaced with successful reads.

Apply explicit READ COMMITTED isolation on SQL Server and PostgreSQL for write transactions. Record SQL Server READ_COMMITTED_SNAPSHOT and related settings; isolation implementations across engines differ, so interpret

pairs within the same engine. Use default durable commits: disable neither SQL Server log durability nor PostgreSQL fsync, synchronous_commit, or full_page_writes. SQLite memory transactions are intentionally nondurable. Optional relaxed-durability tests form a different profile.

Use 15-second command and connection timeouts where supported, a 5-second SQLite busy timeout, and no application retry policy in the controlled baseline. Record provider internal retry behavior and disable it when possible. Counts of timeouts, conflicts, deadlocks, busy/locked results, and rejected work are mandatory. Never omit failed latencies or describe a contention failure as a successful fast operation.

8 ORM and driver parity

Use one fresh DbContext or EntityManager/stateful Session per logical operation. Reuse the application factory, but disable DbContext pooling, application result caches, ORM second-level caches, and ORM query-result caches in the baseline. Do not keep a persistence context alive across the whole run. Ordinary metadata and query-plan caches warm naturally. First-level tracking remains enabled for D01, D02, D03, D05, D06, and D07. D04 has scalar projections. Lazy loading is disabled.

Do not compare EF AsNoTracking with managed Hibernate entities as the primary entity test. Add a separate read-only variant with equivalent output and explicitly documented identity/tracking semantics. Hibernate StatelessSession has different persistence-context behavior; it is a separate variant, not an automatic synonym for no-tracking managed entities. Context pooling and connection pooling are independent controls. [9, 10]

Disable command batching in the baseline where possible: EF MaxBatchSize=1 and Hibernate JDBC batch size=0, with provider-specific verification. Do not use raw execute-many/bulk APIs in that baseline. A batch profile uses the same declared batch size, rows per transaction, commit frequency, and returned data; if the stacks cannot meet the same batching contract, report the mismatch rather than forcing a headline ratio.

For network engines, cap the physical connection pool at 16 per application. Prewarm 16 connections before steady-state timing. Use the same maximum connection count, documented minimum/idle policy, connection lifetime policy, and acquisition timeout. Java may use HikariCP; .NET may use provider pools. Record implementation differences. Do not stack two physical pools accidentally. At 32 application workers, waiting for a 16-connection pool is intentional and measured. For the mandatory SQLite profile use one physical worker connection plus the anchor, and disable hidden pooling if an explicit lease pool is used.

Hold one connection lease for the logical operation on both sides and dispose it after complete result consumption/commit. ORM context construction and disposal are timed. Provider/Open connection lease cost is timed, even if it returns a prewarmed physical connection. Factory initialization and initial physical connection establishment belong in the cold profile.

Use parameterized execution on both raw implementations. Record driver auto-prepare thresholds and statement cache policies; disable optional client auto-prepare for the controlled profile where supported. Do not claim identical SQL Server wire protocols merely because both APIs expose Prepare. A matched explicit-prepared variant may be added. Driver/protocol differences that cannot be removed are part of the stack comparison and must appear in the manifest. For PostgreSQL, verify pgJDBC server preparation thresholds and Npgsql settings explicitly. [11]

9 Complete performance case catalog

Every listed baseline case is mandatory unless marked supplemental. Sizes are fixed inputs, not permission to choose a different easier input for one language. The AI must emit one inventory entry per case, parameter, language, profile, and applicable engine/access cell. All timed code must pass the common oracle. A library comparison must name the libraries, not claim a syntax-only result.

Compute and numeric processing

ID	Operation and inputs	Semantic parity and principal metric
C01	Process 1 million runtime-loaded 64-bit integers through add, multiply, xor and rotate	Explicit modulo-2^64 results; items/s; no compile-time constants
C02	Dense scalar matrix multiplication at N=128 and 512, identical loop order	IEEE binary64; disable contraction differences where feasible; tolerance 1e-9 absolute plus relative; FLOP/s
C03	Price and tax calculation for 100,000 invoices	Values fit C# decimal; Java BigDecimal with declared scales and HALF_EVEN rounding at the same steps; invoices/s
C04	Big-integer modular exponentiation on fixed 2048-bit inputs	Same base, modulus, exponents, signedness and outputs; operations/s
C05	SHA-256 on fixed 1 KiB and 1 MiB byte blocks	Same digest, input reuse and hardware acceleration policy; bytes/s and CPU/byte
C06	Gzip compress and decompress 1 MiB text and incompressible corpora	Fixed format/level where implementations expose it; validate round trip; report output size and throughput separately

C05/C06 measure cryptographic/compression libraries and their native backends. Different compression ratios are a tradeoff, not equal work; do not rank throughput alone unless output sizes are within the declared 1 percent tolerance. C02 gets a separate explicit SIMD/library-optimized variant; a scalar implementation must not compete against one language calling a BLAS library.

Collections and text

ID	Operation and inputs	Semantic parity and principal metric
K01	Scan/sum 1 million primitive long values; sort a fresh copy	Primitive arrays on both sides; copy/setup outside sort timing equally; elements/s
K02	Append, iterate, and remove from a list of 100,000 reference records	Same initial capacity, record fields, identity requirements and removal trace; operations/s
K03	Map insert/lookup/remove for 100,000 ASCII keys; separate long-key variant	Same hit/miss ratios and key equality; specify initial capacity/load policy; operations/s and retained RSS
K04	Set union/intersection of two 100,000-key sets with 50 percent overlap	Same equality and returned members; prebuilt versus build-inclusive variants kept separate
K05	UTF-8 encode/decode and ordinal search over fixed ASCII and Unicode corpora	Validate malformed-input policy and exact decoded values; bytes/s
K06	Parse integers and run a portable regular-expression token extractor over 1 MiB	Same grammar, captures, locale, malformed-input handling and regex reuse; records/s

Do not compare List<int> with Java ArrayList<Integer> as evidence about generic container overhead without labeling representation differences. K01 is the primitive baseline; boxed generics and specialized primitive containers belong in L01 variants. For regex, report compiled/interpreted, timeout, and nonbacktracking choices rather than secretly selecting different algorithms.

Memory and garbage collection

ID	Operation and inputs	Semantic parity and principal metric
M01	Allocate a 64-byte logical reference record graph, consume fields and release	Same logical fields/escape behavior; report actual object sizes, allocations, throughput and RSS
M02	Steady object churn with 1 percent, 10 percent and 50 percent survival	Fixed allocation sizes and retention windows; 120-second runs; throughput and pause distribution
M03	Retain 1 million linked reference records, then run a fixed query trace	Equal reachable data and reference graph; peak/steady RSS and CPU/op
M04	Copy/slice/parse 4 KiB and 1 MiB byte buffers	Allocate-copy baseline; separate pooled/zero-copy variants with equal lifetime guarantees
M05	Fixed mixed allocation service under 512 MiB and 2 GiB process limits	Same offered workload; completed operations, deadline misses, OOM and peak RSS

Record allocation bytes/op using runtime-specific profilers as diagnostic evidence, with their counting boundaries disclosed. Those counters are not automatically interchangeable. Measure OS RSS and total process CPU through the same collector for the comparable resource metrics. Do not force full garbage collections inside timed operations. Forced post-run collection may support a separate retained-heap diagnostic. Runtime pause events must not be equated with end-user latency without service measurements.

Concurrency and asynchronous work

ID	Operation and inputs	Semantic parity and principal metric
A01	Fixed CPU task graph, 100,000 tasks, bounded queues	Same work per task and workers 1/4/8 within the declared CPU budget; throughput and task latency
A02	Producer/consumer pipeline with capacity 1,024 and 1 KiB messages	Equal producers/consumers, backpressure, ordering and checksum; messages/s
A03	Atomic increment versus lock-protected counter	Same shared state, contention, final count and worker counts; increments/s
A04	1,000 and 10,000 in-flight requests to a controlled I/O delay service	Separate equal platform-thread sync and idiomatic C# async/Java virtual-thread profiles; throughput and end-to-end p99
A05	Bounded fan-out with deadline and injected child failure	Same fan-out, cleanup obligations and cancellation outcome; cancellation-to-quiescence latency and leaked work

Use one language-neutral I/O delay server and the same trace, payload, connection cap, and deadlines. Do not compare Task.Delay against Java platform-thread sleeps and call it I/O efficiency. A04's idiomatic profile is required as an additional application result, but stays outside the controlled baseline score. A05 is a correctness and behavior gate first; its speed is diagnostic unless both implementations meet the same cancellation contract.

Serialization and data transformation

ID	Operation and inputs	Semantic parity and principal metric
S01	Serialize and deserialize 1 KiB, 64 KiB and 1 MiB nested JSON documents	System.Text.Json and Jackson as named baselines; identical field mapping, nulls, dates, integer ranges and Unicode behavior
S02	Encode/decode the same Protocol Buffers schema and binary vectors	Pin protoc and generated runtimes; identical packed fields, schema and returned values; bytes/s
S03	Filter/map/group/sort 100,000 records	LINQ versus Java Streams plus an imperative baseline; same materialization and ordering, no I/O
S04	Parse and emit a 10 MiB CSV corpus	Same quoting, escaping, newline, headers and malformed-row behavior; rows/s

For JSON, compare semantic equivalence of parsed outputs; property ordering alone need not affect correctness unless the endpoint contract requires canonical bytes. Make reflection-based and source-generated serializers separate profiles. Do not silently use generated metadata on one side and reflection on the other and label the result library-neutral. Report bytes emitted and allocated, not only time.

File and network I O

ID	Operation and inputs	Semantic parity and principal metric
I01	Sequential read/write of 1 GiB with 64 KiB buffers	Same filesystem and flush contract; buffered OS-cache and durable-flush profiles separate; bytes/s
I02	Seeded 4 KiB random reads over 1 GiB	Same address trace, outstanding request count and cache state; IOPS and latency
I03	Stream 1 MiB messages to a fixed echo server	Same TCP settings, payload, connection reuse and concurrency; throughput and round-trip percentiles
I04	TLS 1.3 handshakes and persistent encrypted transfers	Same cipher/certificate/server, session reuse policy and payload; handshakes/s and bytes/s

A warm page-cache test is not a disk benchmark. An fsync-equivalent durable write is not comparable with a buffered write. Never clear a system-wide cache without documenting the mechanism and restoring the identical state for the pair. Use a separate host when network bandwidth is the target; loopback measures the local stack and must be labeled accordingly.

HTTP services and end to end applications

Use ASP.NET Core and Spring Boot MVC with embedded Tomcat as the named general-purpose baseline frameworks, subject to verified runtime compatibility. Match endpoint behavior and infrastructure; the frameworks' internal work is part of this application comparison. Add a minimal Java server or WebFlux only as a separately named framework/scheduling profile. Publish the exact choice before measurements.

ID	Endpoint	Required response or operation
H01	GET /plaintext	Fixed 32-byte UTF-8 body, equal headers, keep-alive and status

ID	Endpoint	Required response or operation
H02	GET /json and POST /json	S01 schema at 1 KiB and 64 KiB; parse/validate on POST; same output
H03	GET /compute	Fixed C01 subset loaded at runtime; same response and checksum
H04	GET /customer/{id}	D01 through each of the 12 database/access cells; full DTO serialization
H05	GET /order/{id} and POST /order	D03 read and D06 write through all 12 cells; same validation and status; use the engine profile's durability
H06	POST /checkout	Read product prices; validate one customer; insert one order/five lines; update customer balance/version in one transaction; return created ID/total/version

H06 must have a frozen logical statement/transaction contract and data ownership trace, with equivalent rollback on injected failure. It runs against SQL Server and PostgreSQL in both access modes. SQLite at one worker is an additional embedded application case, not a durable checkout claim.

Match HTTP version, request sizes, keep-alive, connection count, compression, TLS, JSON rules, middleware obligations, and logging. Disable body/SQL access logging during measured runs. Perform the same validation and error conversion. An endpoint with authentication enabled cannot compete with an unauthenticated counterpart. An optional auth profile verifies the same JWT algorithm, key, claims and expiration checks.

Use a common external load generator in constant-arrival-rate mode for HTTP and A04. It must keep the offered schedule independent of server response speed, report unsent/dropped arrivals, and record latency from intended arrival to completion where supported. Include queueing time and report overload rather than erasing it through client backpressure. Calibrate the generator and verify it has spare CPU/network capacity. HDR histograms or equivalent raw latency records preserve tail evidence; closed-loop latency alone cannot establish an open-loop service SLO. [14]

Startup and deployment

ID	Scenario	Measurement boundary
P01	Cold process launch to first correct H02 response; separately H04	Fresh application process; dependencies ready; first successful validated response; 30 launches per language
P02	Idle service then fixed 60-second H02 load	Idle RSS after 30 seconds; peak RSS and CPU; identical resource budget
P03	Publish/package the application	Clean dependency-cached build time, artifact bytes, runtime-inclusive deployment bytes, SBOM and license inventory
P04	Start the packaged container or executable	Image already present; container start to readiness and first correct response; record image transfer size separately

Do not include download time for one stack and exclude it for the other. Distinguish cold process, cold file cache, cold host, and cold database. P01's baseline uses a warm OS file cache; a cold-cache profile is separate. Compare

framework-dependent deployment with a comparable JVM/runtime requirement; also report complete runtime-inclusive images. A single-file executable and a JAR can carry different runtime dependencies.

Run P01/P02/P04 separately for matched native AOT variants if supported. Preserve the same HTTP/serialization/DB semantics. Do not silently remove an ORM, reflection behavior, validation, or endpoint to obtain a smaller native executable. Record unsupported features and native build time/peak memory. AOT and JIT results remain separate. [12, 13]

Native interop and language mechanisms

ID	Scenario	Measurement rule
N01	Call the same precompiled C ABI function on 1 and 1,024 integers	C# P/Invoke and Java Foreign Function and Memory API; same native binary, copies and ownership
N02	Traverse a native 1 MiB buffer and return a checksum	Same pinned/off-heap boundary and cleanup obligations; include allocation/free only in the build-inclusive variant
L01	Generic numeric container, boxed references, primitive arrays and value records	Matched reference semantics baseline plus explicitly labeled idiomatic representation variants
L02	Switch/pattern dispatch over the same prebuilt tagged input distribution	Same returned values and branch frequencies; no object construction hidden on one side
L03	Success paths and 0.1/1/10 percent exception paths	Same stack depth, exception construction and stack-trace policy; failures validated
L04	Closure/delegate/lambda invocation and reflection lookup/invocation	Capturing/noncapturing and cached/uncached metadata separated; same target method
L05	Clean, no-op incremental and one-method rebuild of the benchmark applications	Dependency downloads excluded; output verified; 10 independent builds; diagnostic only

N01/N02 have mandatory C ABI paths; an optional JNI alternative is separate. L01-L05 diagnose implementation mechanisms and developer build costs. They have zero default aggregate weight because the primary cases already include many of their costs, and matching semantic/representation choices is nuanced. C# 15 union or closed-hierarchy dispatch may be added to L02 with the Java sealed/tagged alternative; verify actual compiler support and do not infer usability from nanoseconds.

10 Execution profiles and scheduling

The controlled DAL database baseline uses synchronous operations in both languages, fixed workers, no HTTP, and no JSON serialization. Run SQL Server and PostgreSQL at worker counts 1, 8, and 32 with a 16-connection pool. The single-worker result is the designated scored database configuration; 8/32-worker scaling results are mandatory evidence and sensitivity results. Run SQLite at one worker and one leased connection. This makes the primary matrix reproducible without confusing SQLite writer locks with language scheduling.

For compute/collection/serialization microbenchmarks use BenchmarkDotNet and JMH in separate optimized executables, with the same operation boundaries, inputs, state lifetime, process CPU budget, and at least five independent launches. Preserve both harness outputs. Because harnesses differ, obtain score-bearing throughput and OS resource metrics from an additional common fixed-work batch protocol timed externally. Each batch lasts at least one second; freeze the same work count for both languages after pilot calibration. Warm the worker, then use at least five paired independent worker processes and the macro repetition schedule below. Measure only

the worker process, excluding controller/harness parent RSS and CPU. Harness measurements corroborate and diagnose the batch results. Do not equate JMH iteration count with BenchmarkDotNet invocation count. Never put per-item console output, random generation, file import, or network setup into only one timed body. [7, 15]

For macro/DAL cases use at least five independent paired repetitions. Each repetition starts a new application process, warms for at least 60 seconds, and measures for 120 seconds, except the bounded mutation cases below. Extend warmup up to 300 seconds when throughput/JIT/GC behavior has not stabilized. Log warmup windows and the stabilization decision. If it remains unstable, mark it unstable and report the trajectory rather than selecting a favorable window. Long-tail and allocation cases may need longer measurement, chosen in the pilot for both sides.

D05-D08 use the same fixed operation count and trace on both sides, with throughput calculated from actual completion duration. Start with 50,000 operations per repetition and adjust the common count in the pilot to fit the delete population and memory budget while targeting at least ten seconds of work where feasible. Freeze that count for each paired case. Both sides finish the entire trace, with no extra operations by the faster side. This matches data growth and deletion work and prevents an unlimited SQLite insert loop from filling memory. If the required trace cannot fit or lasts too briefly for reliable timing, redesign the common dataset/count before scoring or mark the case blocked. Warmup uses a separate bounded trace, followed by a reset. HTTP mutation tests retain fixed arrival windows and must provision enough trace/data headroom for the frozen offered load.

For each case and parameter, randomize which language runs first using a checked-in schedule seeded with 20261025. Balance AB/BA ordering across the campaign. Run the pair close together on the same host. Do not run every C# case on one day and every Java case on another. Avoid arbitrary sleeps as a cache policy; use explicit readiness checks and measured thermal/storage stabilization.

Restore canonical database contents before each language repetition and after mutation warmup, outside measured time. Keep the runtime and factories warm when restoring after warmup, invalidate affected ORM state, and repeat the same untimed read-preheat sequence on both sides. For SQLite, keep the anchor open and reset/reseed via a control connection outside timing. Verify all logical row counts and versions. Durable write benchmarks commit real changes; rolling back every operation is a different test.

Separate process/runtime cold initialization from data-cache state. Default database results use warm data pages and current statistics. Optional cold-data profiles require an identical restart/cache-reset procedure per engine and remain distinct. Save server uptime, statistics-update actions and cache state before each measured pair.

For HTTP, first pilot a common offered-rate grid. Default SLO is p99 at or below 100 ms with error rate at or below 0.1 percent. Every counted error includes application failures, rejected/unsent arrivals, and deadline misses. Use a 15-second request deadline. Freeze any workload-specific SLO changes before scored runs. Measure both stacks at the same offered rates, including 25/50/75/90/100/110 percent of a common pilot capacity estimate; refine the shared grid symmetrically. Report maximum sustainable successful requests/s meeting the SLO and p99 at a common rate equal to 70 percent of the lower pilot sustainable capacity. Drain or classify all in-flight requests after the fixed arrival window; late completions do not disappear.

11 Campaign stages and runtime budget

Stage	Required work	Permitted conclusion
Preflight	Resolve versions, architecture, licenses, schema/SQL parity, data hashes and resource limits	Compatible or blocked; no performance ranking
Smoke	All case IDs and mandatory cells; small data/short runs; oracle and schema validation	Implemented and correct at smoke scale

Stage	Required work	Permitted conclusion
Pilot	Representative sizes/rates, all providers, mutation trace sizing, load calibration and warmup assessment	Feasible settings and duration estimate
Full	Frozen complete inventory; five paired macro repetitions; micro harness launches; startup/build repetitions	Measured comparisons with uncertainty
Diagnose	Plans, allocation/GC traces, profiles and explanations after a result anomaly	Causal hypotheses and separately tested variants

The controller must list the exact generated case count and estimated wall time before execution. A comprehensive campaign is a multi-hour or multi-day task. For example, just the network DAL portion has 2 engines x 2 access modes x 8 workloads x 3 worker counts x 2 languages x 5 repetitions = 960 measured process repetitions. If all use 60 seconds warmup plus 120 seconds measurement, that is 48 hours before reset/startup overhead. SQLite adds 160 repetitions, approximately eight hours at that schedule. Actual duration changes with bounded mutation traces, extended warmup, and setup overhead. HTTP and the non-database suite add further time. These are arithmetic estimates, not measured execution times.

The AI may offer a compact subset, but must name its coverage and must not label it the full suite. Support checkpoint/resume keyed by immutable case and input hashes. Reuse a completed result only when its environment, code, parameters, and data hashes exactly match. A changed implementation starts a new campaign ID.

12 Metrics and evidence

Metric	Definition and required boundary
Successful throughput	Correct completed logical operations divided by measured wall time; not SQL statements or attempted requests
Service latency	Request scheduled arrival to final response; include queueing and timeouts; report p50/p95/p99 and maximum
DAL operation latency	Before connection lease/context creation to complete materialization or commit and disposal; label closed-loop service time
CPU per operation	Process user plus system CPU seconds per correct operation; network DB CPU also reported separately and as total system CPU
Memory	OS process RSS sampled at 100 ms or a documented common interval; peak and steady; container/cgroup peak and DB memory separately
Allocation and GC	Allocated bytes/op, collection counts, pause times and live heap using named runtime-specific tools; diagnostic boundaries disclosed
Database cost	Statements, round trips, rows read/returned/modified, log/WAL bytes, physical/logical I/O, locks, waits, plans, DB CPU and memory
Startup and packaging	Launch/readiness/first response, publish/build time, artifact and full deployment size, native build memory where applicable

Metric	Definition and required boundary
Reliability	Attempts, successes, failures by reason, conflicts, timeouts, dropped arrivals, retries, incorrect outputs and leaked work

Use monotonic clocks for durations and UTC timestamps for chronology. Preserve clock resolution, histogram units/range/significant digits, measured intervals, and histogram overflow count. Record p99.9 only with at least 100,000 valid observations and an explicit warning about tail uncertainty; do not infer confidence from a large request count alone.

Primary timings use minimal common OS sampling and equivalent result consumption. Heavy profilers, SQL statement logging, execution plans, per-operation stack traces and allocation analysis run separately. Measure instrumentation overhead with an empty/known-work control and report it. Do not subtract speculative overhead from only one language.

Save a result per process repetition, never only a consolidated chart. Required evidence includes source commit/hash, complete commands, stdout/stderr, native versions, database/server settings, canonical manifests, trace hashes, SQL/plan diagnostics, harness JSON, OS samples, latency histograms, failure records and validation results. Store connection secrets separately and redact them from evidence.

13 Statistical analysis and decision rules

The independent unit is a process repetition or benchmark-harness launch, not every request inside a long-lived process. Report every repetition, median, range and run-level variability. For paired macro cases compute the per-pair C#/Java throughput ratio, or Java/C# ratio for lower-is-better metrics, so a ratio above one favors C#. Use the geometric mean of paired ratios and a paired bootstrap 95 percent confidence interval, resampling whole pairs 10,000 times with a recorded seed. Five pairs provide limited precision; expand both sides symmetrically if necessary.

For an exploratory result, label an advantage only when the confidence interval excludes one and the ratio differs from one by at least 5 percent. Otherwise report inconclusive or practically similar; a failed significance threshold does not prove equivalence. A formal equivalence claim requires a preregistered equivalence test and bounds. For many simultaneous cases, publish all intervals and qualify them as exploratory; a report asserting family-wide significance must apply and disclose a multiple-comparison procedure.

Never average p99 values across requests or merge histograms from different offered rates. A per-repetition p99 can be summarized across repetitions, with that boundary explicit. A percentile computed from all pooled requests answers a different question and must be labeled. Do not discard outliers automatically. Exclude a run only under preregistered invalidation rules, preserve its evidence, and rerun the whole pair.

Measure saturation curves, not just peak throughput. If a language completes more requests while timing out substantially more, that is not an unconditional win. Zero measured errors is not proof of zero failure probability. Report crash, OOM, invalid output and unsupported cases as such; do not manufacture timing ratios from them.

14 Weighted performance ranking

This proposed model ranks the measured implementation portfolio, not language expressiveness or ecosystem quality. Its weights are explicit architectural judgments, not objective scientific constants. Freeze weights before the scored campaign; publish unweighted cases and sensitivity results alongside the score. Popularity, employment demand and IDEs contribute zero.

Performance domain	Weight	Default metric weights within the domain
Compute C01-C06	12 percent	Throughput 70; CPU/op 30
Collections K01-K06	10 percent	Throughput 60; CPU/op 25; peak RSS 15
Memory M01-M05	12 percent	Throughput 40; peak RSS 40; CPU/op 20
Concurrency A01-A04 controlled	12 percent	Throughput 70; CPU/op 30; cancellation A05 is a gate/diagnostic
Serialization S01-S04	8 percent	Throughput 60; CPU/op 25; peak RSS 15
File and network I01-I04	8 percent	Throughput 70; CPU/op 30
HTTP H01-H06	14 percent	SLO-qualified throughput 50; common-rate p99 25; CPU/op 15; peak RSS 10
Databases D01-D08	18 percent	Single-worker throughput 60; total system CPU/op 25; application peak RSS 15
Startup and deployment P01-P04	4 percent	First correct response P01 50; idle RSS P02 25; runtime-inclusive image size P04 25
Native interop N01-N02	2 percent	Throughput 70; CPU/op 30

Weights total 100 percent. Within a domain, case IDs have equal weight; parameter variants split their parent case weight equally. In HTTP H05 splits its read/write endpoint weight equally; H02 splits GET/POST and sizes equally. Cases that expand across database cells split engine weight SQL Server 40 percent, PostgreSQL 40 percent, SQLite 20 percent where all three apply, then split ORM/raw 50/50. H06 uses SQL Server/PostgreSQL 50/50; its SQLite diagnostic has zero baseline weight. D08's fixed mix is a workload in its own right, with the same case weight as other D cases.

The database domain applies those same engine weights and ORM/raw split. Aggregation uses normalized within-engine C# versus Java results; never average raw SQLite and SQL Server throughput. SQLite therefore contributes 3.6 percentage points of the full performance index through the database domain. Publish separate SQL Server, PostgreSQL, SQLite, ORM and raw scores so this choice stays visible. Run a required sensitivity view with SQLite weight zero and network engine weights 50/50.

Score only the designated S dataset, JIT controlled profile, uniform trace, warm steady state, and designated concurrency/load for each case. For A01-A03 use four workers; for A04 controlled use 1,000 in flight with equal platform-thread budgets verified in the pilot. For I03/I04 use the same pilot-frozen concurrency. H01-H06 use their defined SLO throughput/common-rate latency. P01 scores the H02 startup case; H04 startup remains separately reported. P03 build-time/license outputs and L01-L05 are diagnostics with zero default weight. Additional sizes, workers, native executables, tuning and stable-runtime tracks never gain accidental extra weight.

For a higher-is-better metric x, assign each language a 0-5 score: 5 times x divided by the larger of the two observed values. For a lower-is-better metric x, use 5 times the smaller value divided by x. Use paired aggregate estimates from admitted repetitions; zero/undefined resource measurements require diagnosis, not division by zero. Equal results give both sides 5. The weighted performance index is 20 times the weighted mean of these 0-5 leaf scores, yielding 0-100. It is a best-relative index for this two-stack campaign; it is not a percentage of theoretical maximum performance and does not imply constant meaning across unrelated campaigns.

Example only, not measured: if one implementation performs 1,200 correct operations/s and the other 1,000, their throughput scores are 5 and 4.167. In a leaf weighted at 2 percent of the total, the index contribution differs by approximately 0.333 points. A narrow win in a tiny case cannot overwhelm the entire result simply by counting as one win.

Use the correctness, parity and completeness gates before scoring. If a mandatory cell is unsupported, invalid, or missing, the full index is unavailable. Publish coverage and optional explicitly named partial indices with their fixed selected scope; never silently redistribute missing weights. A measured SLO failure is a behavior result, not missing evidence: its sustainable throughput is the highest positive rate actually satisfying the SLO, and remains unavailable if no such rate was established. Do not invent a zero-rate measurement.

Bootstrap whole paired repetitions to provide an aggregate uncertainty interval where data supports it; preserve covariance between metrics from the same run and do not imply cross-case pairing unless the campaign actually used common block IDs. Mark unresolved aggregate uncertainty explicitly. Publish sensitivity to equal domain weights, a database-heavy profile, a latency-heavy profile, SQLite exclusion, and the stable corroboration track. Choose sensitivity weights before examining the final ranking.

To connect this plan to the earlier overall language/platform ranking, retain its 8 percent benchmark category rather than replacing the other 92 percent with runtime measurements. Convert the performance index to that category's 0-5 score by dividing by 20; its contribution to a 100-point overall ranking is 8 times index/100. The unmeasured category remains unscored until the required evidence exists. Benchmark failures do not establish poorer syntax, smaller ecosystem, or more restrictive licensing.

15 Hard gates and common invalid comparisons

Reject a paired result when outputs differ, the dataset/trace hashes differ, one side performs fewer required operations, only one transaction durably commits, required parameters/columns differ, cache/tracking semantics are mismatched, resource limits differ, or one implementation silently retries. Also reject missing result consumption, exhausted traces, hidden N+1 loading, unreported errors, floating dependency versions, load-generator saturation, histogram overflow, and an unintended production connection.

Do not reject a run merely because it makes a favored implementation look slow. A real OOM or unsupported provider remains a reportable capability result. A host failure or thermal interference invalidates the matched pair under a published rule, not an individual inconvenient observation.

Invalid shortcut	Required comparator
EF Core InMemory versus H2	SQLite in memory on both sides, with native engine parity
Tracked entity graph versus scalar/raw reader only	Same returned graph and result consumption
ORM read-modify-write versus one raw atomic update	Same read, write, version check and commit contract
One language against SQL Server and the other against PostgreSQL	Pair languages within each database engine

Invalid shortcut	Required comparator
C# async I/O versus thousands of Java platform-thread sleeps	Same real I/O server; controlled and idiomatic scheduling tracks
C# struct array versus Java boxed object list without qualification	Primitive/reference semantic baseline; representation variant named
Default JVM warmup versus already-hot .NET process	Independent process launches and recorded stabilization on both sides
Native executable versus JIT process in one headline	Matched deployment tracks and complete dependency boundaries
Only successful requests included in latency	Errors, dropped arrivals and deadlines reported with scheduled load
A chart containing invented placeholders	Explicit not measured status and no numerical ranking

16 Repository and AI deliverables

The AI must implement the following logical layout. Names are a command/interface contract; the implementation may add files but must preserve one authoritative source for shared inputs.

```
benchmark/
  README.md
  specification.md
  config/campaign.json
  config/versions.lock.json
  config/weights.json
  config/cases.json
  data/generate.py
  data/manifest.json
  schema/sqlserver.sql
  schema/postgresql.sql
  schema/sqlite.sql
  queries/<engine>/<case>.sql
  dotnet/
  java/
  native/bench.c
  controller/benchmarkctl.py
  validation/
  reporting/
  infrastructure/compose.yaml
  evidence/<campaign_id>/
  results/<campaign_id>/
```

Provide a README with prerequisites, exact install/build commands, supported platforms, resource estimates, test boundaries, credentials configuration, reset safeguards, EULA/license notes, and troubleshooting. Include dependency locks, container digests, an SBOM, configuration schema, command help and a deterministic example generator run. Do not bundle a proprietary benchmark suite or unredistributable database image.

Expose these commands with JSON output and nonzero failure exit codes. They are required future interfaces, not commands implemented by this document.

```
python controller/benchmarkctl.py preflight --config config/campaign.json
python controller/benchmarkctl.py validate --all
python controller/benchmarkctl.py inventory --format json
python controller/benchmarkctl.py run --stage smoke --all
python controller/benchmarkctl.py run --stage pilot --all
python controller/benchmarkctl.py run --stage full --all --dry-run
python controller/benchmarkctl.py run --stage full --all --resume
python controller/benchmarkctl.py report --campaign <id>
```

Support selectors for language, engine, access mode, case, dataset, profile, workers and repetition. Run/reset only controller-created allowlisted databases. Validation must precede measurement automatically. Report unsupported cases without an infinite retry loop. Checkpoint results atomically so an interruption does not erase successful repetitions.

The final AI delivery includes both compilable implementations; shared fixtures and oracles; all 12 database cells; the full non-database catalog; parity tests; load-controller code; scripts and configuration; raw result JSON/CSV and histograms if executed; a human report; exact reproduction commands; coverage; and a clear implemented/validated/measured/admitted status per case. If execution was unavailable, deliver working implementation and instructions while marking performance results not measured.

17 Machine readable result contract

Each repetition result must contain the following field groups. Use null for unavailable metrics plus a reason; never use a fabricated zero. A machine-readable JSON Schema must be generated and enforced by the implementation AI.

Field group	Required content
Identity	campaign_id, case_id, parameter_id, pair_id, block_id if applicable, repetition, language, access, engine, profile, status
Provenance	UTC start/end, source hash, exact runtime/compiler/ORM/driver/DB versions, image/native hashes, command, host manifest hash
Input	dataset name/hash, trace hash/seed, schema hash, SQL diagnostic hashes, payload bytes, result contract version
Load	workers, pool limit, offered rate, actual sent rate, scheduled/unsent/rejected/completed counts, warmup/measurement duration
Outcome	correct successes, categorized failures, correctness verdict, final state checksum, timeouts/conflicts/retries
Performance	successful ops/s, latency histogram path, p50/p95/p99, CPU seconds, CPU/op, RSS samples/peak, total-system metrics
Diagnostics	GC/allocation paths, SQL command/roundtrip counts, plans, DB I/O/waits, profiler overhead classification
Admission	admitted boolean, exclusion/unsupported reason, gate results, coverage weight and evidence paths

The report generator must reject conflicting units, missing required hashes, negative durations, successes exceeding attempts, histogram/count inconsistencies, or attempts disappearing between scheduling and

completion. Histograms for successful and failed/timed-out operations may be separate, but the report must disclose both and compute SLO admission using all scheduled work. Unsent work has no measured completion time: count it as a failure and mark its latency unavailable or right-censored at the deadline, never zero. Distinguish successful-response percentiles from all-attempt latency summaries, and do not label censored deadlines as exact completion times.

18 Acceptance checklist

1. Both languages compile on the resolved requested track, or blocked cells and alternative tracks are explicit.

2. SQL Server, PostgreSQL and true in-memory SQLite each have ORM and raw implementations in both languages.

3. D01-D08 have identical logical semantics, canonical input, returned data, transaction and validation contracts.

4. Every C/K/M/A/S/I/H/P/N/L catalog entry has an implementation or an explicit unsupported reason; optional profiles are not confused with baseline coverage.

5. Correctness checks include expected failures, rollback, optimistic concurrency, output consumption and complete committed state.

6. Version locks, native SQLite identity, schema/trace hashes, resource limits, SQL plans and pool policies are recorded.

7. Cold/warm, sync/idiomatic, JIT/AOT, raw/ORM and embedded/network engine results remain identifiable.

8. Full measurements use independent repeated processes, balanced paired ordering, calibrated load and preserved raw evidence.

9. No popularity, employment demand, IDE assessment or user preference influences measurement or score.

10. Weighted results follow frozen weights, coverage gates, uncertainty rules and published sensitivity profiles.

11. Reproduction commands work from a clean checkout; dry-run estimates the complete campaign rather than launching it unexpectedly.

12. The final report clearly distinguishes not implemented, unsupported, validated, measured, invalid and admitted results.

19 Sources and interpretation

Sources below support tool behavior, compatibility boundaries and measurement cautions. Dataset sizes, case selections, durations, thresholds, SLOs and weights are design choices in this plan, not claims that a vendor prescribed them. Recheck moving documentation against pinned versions during implementation. Sources were consulted on 25 October 2026. No third-party leaderboard results are imported into this plan's ranking.

[1] Microsoft Learn. What's new in C# 15. Documents .NET 11 linkage and preview status. Source 1

[2] OpenJDK. JDK 27 and GA builds. Establishes the Java version/release track. Source 1 | Source 2

[3] Hibernate ORM. Dialect support and community SQLiteDialect source. Establishes the different community support boundary; exact workload compatibility still requires testing. Source 1 | Source 2

[4] SQLite. In-memory databases. Explains private connections, named memory databases, same-process sharing and connection lifetime. Source 1

[5] Microsoft Learn. EF Core testing and testing without the production database. Explains limitations of substitute providers and discourages the InMemory provider as a database test substitute. Source 1 | Source 2

[6] Microsoft Learn. Microsoft.Data.Sqlite async limitations. Documents synchronous execution of async ADO.NET methods. Source 1

[7] BenchmarkDotNet. Good practices. Release builds, environment boundaries, dead-code elimination and host effects. Source 1

[8] Microsoft Learn. EF Core efficient querying. Indexes, projection and related-data loading. Source 1

[9] Microsoft Learn. EF Core advanced performance topics. Distinguishes context pooling from driver connection pooling. Source 1

[10] Hibernate ORM. User Guide and StatelessSession API. Persistence-context, batching and stateless behavior. Source 1 | Source 2

[11] PostgreSQL JDBC. Server prepared statements; Npgsql basic usage. Parameterized/prepared execution and pooling need explicit policy. Source 1 | Source 2

[12] Microsoft Learn. Native AOT overview and ASP.NET Core support. Native deployment has supported-feature boundaries. Source 1 | Source 2

[13] GraalVM. Native Image. Describes ahead-of-time compilation and native executable deployment. Source 1

[14] HdrHistogram. Project documentation. Histogram precision and preservation of latency distributions. The load-scheduling rules in this plan are additional experiment requirements. Source 1

[15] OpenJDK JMH and .NET performance microbenchmark design guidelines. Harness and design references. Source 1 | Source 2

TechEmpower's public test definitions are an optional reference for HTTP plaintext, JSON, database and template workloads; if its suite is run, retain its exact revision, rules, licenses and workload identity. Its public rankings are not interchangeable with this pinned C# 15/Java 27 campaign. Source 1